Executive Summary

This report presents the results of a comprehensive security assessment conducted on the application, focusing on authentication, authorization, input validation, session management, and configuration controls. The objective of this testing was to identify potential vulnerabilities that could compromise user data, system integrity, or access control mechanisms. Overall, the application demonstrates strong security practices in several areas, including input sanitization, access control enforcement, and session timeout handling. However, certain weaknesses—most notably related to session hijacking—highlight areas that require further attention.

The findings are categorized by severity to help prioritize remediation efforts. While many tested components passed expected security standards, the presence of exploitable vulnerabilities indicates that continued hardening is necessary before deployment in a production environment.

Testing Methodology

The security testing process followed a structured approach combining both manualtesting techniques and a test.py script for simplicity. Each component of the application was evaluated against expected security behaviors, including authentication mechanisms (e.g., password policies and brute-force protections), authorization controls (horizontal and vertical privilege enforcement), and input validation safeguards (such as protection against XSS, command injection, and path traversal attacks).

Testing involved simulating real-world attack scenarios, including unauthorized access attempts, manipulation of URL parameters, and injection of malicious inputs. Session management was evaluated through token handling, timeout enforcement, and attempts to replicate session hijacking conditions. Additionally, configuration settings such as HTTPS

enforcement, security headers, and debug modes were reviewed to ensure adherence to best practices.

Findings

Description

Each identified vulnerability is documented with a clear explanation of the issue, including where it occurs within the application and how it deviates from expected secure behavior. This provides context for understanding the root cause of the issue.

Impact

Session hijacking through obtaining cookies has been the only possible way to perform vertical and horizontal privilege escalation that we have identified in our system. However in real life, obtaining cookies from a user is far more complicated. The impact of this unlikely scenario is high, but at last it's hard to replicate in real life due to browsers usually protecting cookies.

Steps to Reproduce

We only found one vulnerability in our application. To reproduce it one actor must activate the Developer Tools and enter the other actor's session key in the cookies section. This is a high risk vulnerability because the attacker now has access to the victim's documents and contacts. If the victim happens to be an administrator, the attacker can successfully perform privilege escalation.

Proof of Concept

Where applicable, screenshots were provided as evidence to demonstrate the exploitability of the vulnerability. Because we only have one vulnerability with the session hijacking using cookies, don't expect more screenshots of our other methods.

Remediation Recommendations

Session hijacking through cookies can be avoided if we add some IP context whenever an attacker is trying to enter another session, effectively avoiding other IP addresses to enter a session. Devices like cellphones and laptops change their IP addresses constantly because their owners move around a lot. This solution could be secure in some instances, but for mobile devices it could cause a burden on the user due to constantly logging in and out, which affects the availability of the session.

Conclusion

The security assessment indicates that the application has a solid foundation in several key areas, particularly in enforcing access control and preventing common input-based attacks. Most tested functionalities behaved as expected, demonstrating adherence to secure development practices. However, the identification of vulnerabilities such as session hijacking underscores the importance of strengthening session security and safeguarding sensitive tokens.

Addressing the identified issues will significantly improve the overall security posture of the application. Continued testing, along with the implementation of recommended mitigations, is essential to ensure resilience against evolving threats. With these improvements, the application will be better prepared for secure deployment and real-world use.

Checklist

Authentication Testing

- ☑ **Brute force protection:**
    - ☐ Expected: Invalidate all wrong password inputs

- ☑ Actual: error "**Your credentials don't match our bear records. You have an invalid username or password.**"

- ☑ **Password complexity enforcement:**
  - ☐ Expected: "reject pas2 as password for new user"
  - ☑ Actual: **"Password must be at least 12 characters."** IP bans user after too many failed attempts
- ☑ **Session management:**
  - ☑ Expected and Actual: Session expires after 30 minutes and asks user to login again
- ☑ **Logout functionality:**
  - ☐ Expected: Fully implemented Logout button
  - ☑ Actual: User's session expires after login out
- ☑ **Password reset security:**
  - ☐ Expected: Password validation scripts must be used for a password reset. User must know previous password
  - ☑ Actual: Works fine, user must know previous password and craft new password with proper length and sophistication requirements

Authorization Testing

- ☑ **Horizontal privilege escalation [Passed]**
  - ☐ Expected: User/Guest cannot view/delete/modify documents from other users if item hasn't been shared with them

- ☑ Actual: Error 403 **“Sorry, you don't have permission to access this resource”**

☑ **Vertical privilege escalation:**

- ☐ Expected: Only admins can manage users and therefore promote new admins and demote users to guests
- ☑ Actual: Error 403: “**Access Denied”**

☑ **Direct object reference:**

- ☐ Expected: User cannot view/delete/modify documents from peers who haven’t shared it with him, from URL
- ☑ Actual: Error 403 **“Sorry, you don't have permission to access this resource”**

☑ **Forced browsing**

- ☐ Expected: any URL pattern should be denied for users without access to a specific doc id
- ☑ Actual: /download/doc_id, /share/doc_id, /delete/doc_id, /view/doc_id return an Access error for not appropriate users

Input Validation Testing

☑ **XSS attempts:**

- ☑ Actual: Every input is flagged because of input sanitizing. Script is not executed

☑ **Path traversal:**

- ☑ Actual/Expected: Users without permissions to a file cannot access files through links like /download/doc_id, /view/doc_id,

☑ **Command injection:**

- ☑ Actual/Expected: Command inputs are sanitized instead of returning something in the website to the attacker. Shell scripts don't affect the website

☑ **File upload vulnerabilities:**

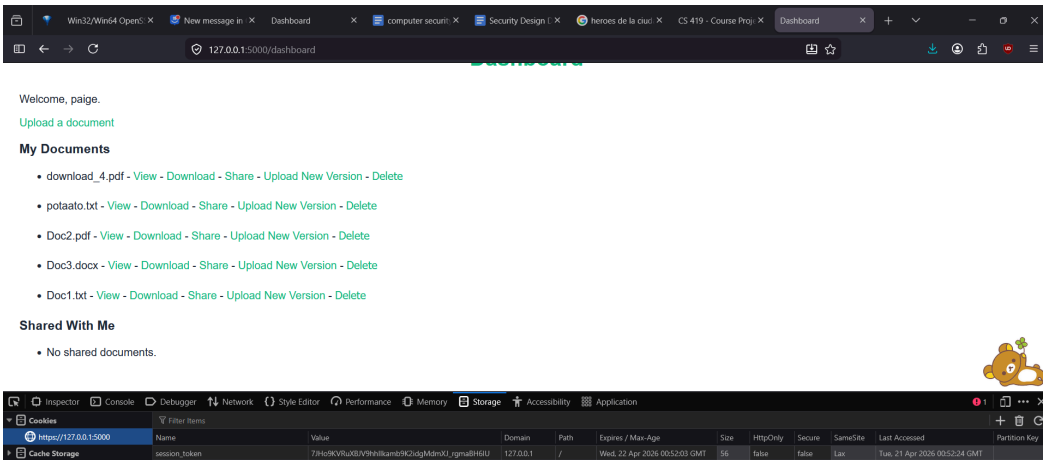
- ☐ Expected: app should have a robust resolution to wrong file types being uploaded
- ☑ Actual: app performs mime type checks and rejects files that are not .pdf, .docx or .txt. Error: **"Invalid file type."**

Session Security Testing

☑ **Session fixation**

- ☑ Actual: Set HTTPOnly cookie attribute

☑ **Session hijacking**



- ☐ **Vulnerability[High]:** We put Paige's cookies in Smila's session and were able to hijack her session. We took all the proper precautions with timing the sessions and rotating session keys, but at last this is an inevitable scenario once an attacker has access to your cookies. In the real world browsers take precautions so your session tokens are not exposed to other users.

- ☑ **Concurrent session handling**
  - ☑ Actual: we validate sessions through tokens created
- ☑ **Session timeout**
  - ☐ Expected: User's session times out after 30 minutes
  - ☑ Actual: User's session times out after 30 minutes, asks user to login again

Configuration Testing

- ☑ **Security headers:**

```
princesspaige@MacBook-Pro ~ % curl -k -I https://127.0.0.1:5000
HTTP/1.1 200 OK
Server: Werkzeug/3.1.7 Python/3.14.3
Date: Tue, 21 Apr 2026 00:21:30 GMT
Content-Type: text/html; charset=utf-8
Content-Length: 593
Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect
-src 'self'; frame-ancestors 'none'; base-uri 'self'; form-action 'self'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: geolocation=(), microphone=(), camera=()
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Cache-Control: no-cache, public
Vary: Cookie
Connection: close
```

Cookies

Name	Value	Domain	Path	Expires	Size	Secure	HttpOnly	SameSite
session_token	7JHo9KVRuXBJV9hhIlkamb...	127.0.0.1	/	4/20/2026, 8:53:24 PM	56 B	✓	✓	Strict

- ☑ Expected/ Actual Headers follow assignment guidelines

- ☑ **TLS/SSL configuration**
  - ☑ Actual: Site is loaded at https://, can check by running curl -k -I https:/127.0.0.1:5000
- ☑ **Error handling**
  - ☑ Handles wrong passwords and users accessing forbidden sites.
- ☑ **Debug mode disabled**
  - ☑ Expected/ Actual: App doesn't crash even with the debug mode disabled